

Digital Professor: Interaction Methods

Sep. 18th Update



Jonathan Ma

Prototype Review

What got done

Working local web application

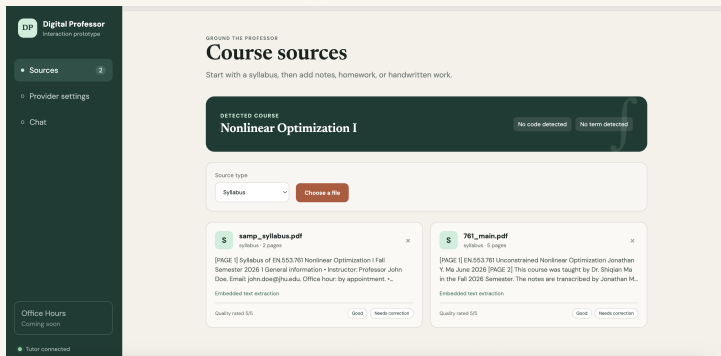
- ▶ Three-tab interface: Sources, Provider Settings, Chat
- ▶ Syllabus-first course setup
- ▶ Course-grounded tutoring with rendered math and code
- ▶ Validated source citations and metadata-only evaluation logs

Four interaction paths

- ▶ Typed questions and equations
- ▶ Document and image ingestion
- ▶ Live sketchpad for handwriting or stylus input
- ▶ Recorded voice converted to editable text

Result: an end-to-end prototype that can be demonstrated on a computer, phone, or tablet.

Demo evidence: course setup and ingestion



Worked well

- ▶ PDF upload was extracted in full
- ▶ Two PDFs were ingested and correctly classified as notes or syllabus
- ▶ Source method and quality controls are visible

Did not work yet

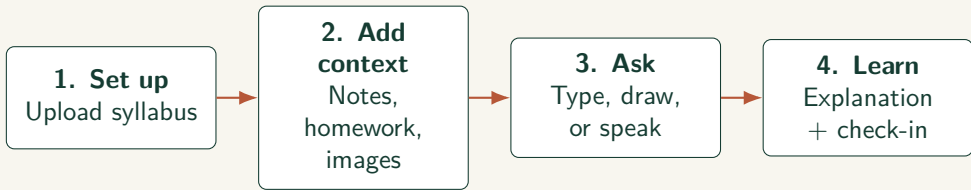
- ▶ Course code and term were not detected, sometimes Names cannot parse
- ▶ Internal page markers appear in the preview
- ▶ Metadata still needs an edit-and-confirm step

[illegible]

- ▶ Model, source exposure, handwriting, and voice controls are centralized
- ▶ API key status is visible without exposing the key
- ▶ Professor behavior is editable without code changes

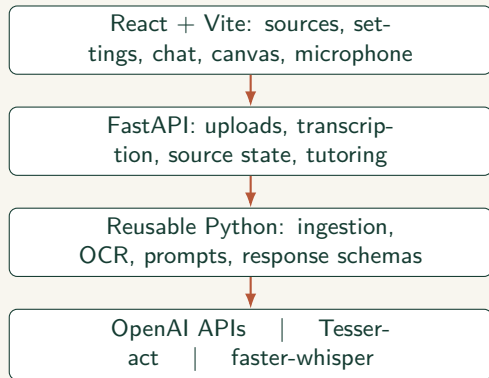
- ▶ Settings require significant scrolling
- ▶ Token rates must be entered manually for cost estimates
- ▶ Some listed models may not support tutoring output

Student experience in the prototype



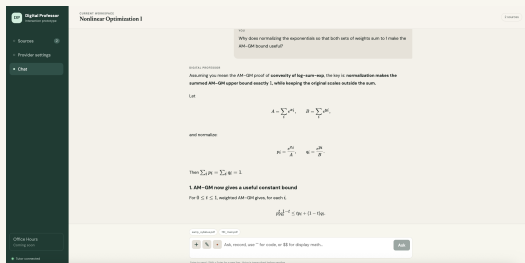
- ▶ The syllabus supplies a proposed course name, code, and term.
- ▶ Uploaded material becomes context for later chat requests.
- ▶ Transcriptions and recognized handwriting remain visible for review.
- ▶ The tutor explains its assumptions and asks whether the student follows.

Prototype architecture



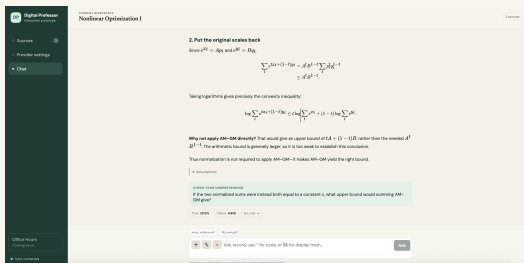
- ▶ Browser-based and currently run on localhost
- ▶ API key stays on the backend
- ▶ Research code is isolated under `research/interaction-methods`
- ▶ Tutoring, explanation, and review instructions live in a configurable skills file
- ▶ In-memory state keeps iteration simple

Demo evidence: tutoring chat



Worked well

- ▶ Course-specific question produced a worked response
- ▶ Display equations rendered cleanly
- ▶ Assumptions, comprehension check, source chips, and metrics are present



Needs improvement

- ▶ Answer is long and requires substantial scrolling
- ▶ Example latency is about 20 seconds
- ▶ Estimated cost is blank until token rates are configured
- ▶ Citation visibility and correctness need live-demo validation

Document ingestion

Implemented pipeline

1. Accept PDF, text, Markdown, $\text{T}_\text{E}\text{X}$, PNG, JPEG, and WebP
2. Extract embedded PDF text locally with PyMuPDF
3. Detect image-only or low-text PDFs
4. Render pages and route them to visual recognition
5. Store normalized text as chat context

Recognition comparison

- ▶ OpenAI vision for handwriting and mathematical notation
- ▶ Tesseract as a zero-API-cost OCR baseline
- ▶ Per-request method and elapsed time recorded

Current gap: citations preserve available page markers, but citation correctness and retrieval quality are not yet formally evaluated.

Live writing and handwriting recognition

What works

- ▶ Sketchpad opens directly from the chat composer
- ▶ Mouse, touch, Apple Pencil, and compatible stylus input
- ▶ Adjustable pen width, clear, cancel, and upload controls
- ▶ Canvas exports a PNG into the existing ingestion pipeline
- ▶ Works across desktop and local-network mobile testing

What this lets us test

- ▶ Natural handwritten questions
- ▶ Ambiguous symbols and notation
- ▶ Equations that are difficult to type
- ▶ OpenAI vision versus Tesseract output

Known limitation: Tesseract is designed mainly for printed text and does not reliably reconstruct 2-D math as $\text{T}_{\text{E}}\text{X}$.

Simple voice recording and transcription

Interaction

- ▶ Start and stop recording from the chat composer
- ▶ Convert speech to editable text before submission
- ▶ Use a file/capture fallback when mobile browsers block recording
- ▶ Accept common audio formats up to the demo upload limit

Testable providers

- ▶ OpenAI: GPT Transcribe, GPT-4o Transcribe, mini, and Whisper-1
- ▶ Local faster-whisper: tiny, base, and small
- ▶ Report provider, model, and transcription time

Known limitation: microphone capture on a phone generally requires HTTPS; plain LAN HTTP uses the recording/file fallback.

Source exposure and evaluation telemetry

Source exposure

- ▶ Provider setting shows or hides citations in answers
- ▶ Structured citation: validated source ID, filename, optional page, and supporting basis
- ▶ Unknown or fabricated source IDs are removed by the backend
- ▶ Page markers are preserved during ingestion when available

Privacy boundary: logs contain metrics and identifiers; they exclude source contents, transcripts, questions, and answers.

Research telemetry

- ▶ JSONL log plus a read-only session metrics endpoint
- ▶ Latency, tokens, configured cost, provider, and model
- ▶ Equation-confidence proxy and student quality rating
- ▶ Edit distance and correction rate for voice drafts
- ▶ Citation count and source-exposure state

What still needs to get done

Area	Next deliverable	Why it matters
Evaluation	Fixed test set, ground truth, accuracy, latency, and cost comparison	Select providers with evidence
Grounding	Chunking, retrieval, citation evaluation, and source previews	Make answers auditable
Course setup	AI-assisted syllabus extraction with editable metadata	Reduce setup friction
Persistence	Database, course workspaces, chat history, and file storage	Support repeat use
Product safety	Authentication, privacy controls, retention rules, moderation	Prepare for real students
Accessibility	Keyboard paths, captions, screen-reader checks, contrast testing	Support diverse learners

Blockers and risks

Technical blockers

- ▶ Local OCR/transcription tools require optional native packages and model downloads
- ▶ Mobile microphone access needs trusted HTTPS
- ▶ Provider model access and behavior vary by API account
- ▶ Current state disappears when the backend restarts

Research risks

- ▶ Handwriting and spoken math remain highly ambiguous
- ▶ A fluent answer may still be mathematically wrong
- ▶ More agents add cost and failure points without proven need
- ▶ Student data introduces privacy and consent requirements

Mitigation direction: retain editable intermediate outputs, expose assumptions, cite sources, and evaluate every modality on the same tasks.

1 Measure the current prototype

Next

- Run identical document, handwriting, equation, and audio samples across providers
- Add ground truth and scoring to the metrics already captured

2 Make course grounding dependable

- Add retrieval, validate page citations, editable course metadata, persistence, and tests

3 Pilot a secure public web application

- Add HTTPS, authentication, storage, privacy controls, and observability

4 Prototype Office Hours after chat evaluation

- Real-time voice, optional video/avatar, interruption handling, transcript, and shared canvas

5 Add manager-worker agents only where evaluation shows value

- Candidate workers: document ingestion, math handling, tablet input, and transcription

Recommended next milestone

Deliver a measured interaction demo

One student task should move through typed input, a source document, handwriting, and voice while the prototype records recognition quality, correction effort, response quality, latency, and cost.

- ▶ Demonstrates the complete value proposition without committing to Office Hours architecture
- ▶ Produces evidence for provider and model selection
- ▶ Reveals whether specialized agents are needed
- ▶ Creates a clear acceptance gate for deployment work

Questions and prototype walkthrough